

Laboratorio No. 8 — Respuestas

Teoría de la Computación — Análisis de complejidad

Ejercicio No. 1

```
void function (int n) {  
    int i, j, k, counter = 0;  
    for (i = n/2; i <= n; i++) {  
        for (j = 1; j+n/2 <= n; j++) {  
            for (k = 1; k <= n; k = k*2) {  
                counter++;  
            }  
        }  
    }  
}
```

a) Complejidad de tiempo

Se analiza cada ciclo de afuera hacia adentro:

- **Ciclo i:** va de $n/2$ a n con paso 1, así que ejecuta $n - n/2 + 1 = n/2 + 1$ iteraciones $\rightarrow \Theta(n)$.
- **Ciclo j:** la condición $j + n/2 \leq n$ equivale a $j \leq n/2$, y j parte de 1, así que ejecuta $n/2$ iteraciones $\rightarrow \Theta(n)$.
- **Ciclo k:** k se duplica en cada paso (1, 2, 4, 8, ...). Tras m iteraciones vale 2^m ; el ciclo termina cuando $2^m > n$, es decir, $m = \lfloor \log_2 n \rfloor + 1$ iteraciones $\rightarrow \Theta(\log n)$.

Los ciclos son independientes (ninguna cota depende de la variable de otro ciclo), por lo que el trabajo se multiplica:

$$T(n) = (n/2 + 1) \cdot (n/2) \cdot (\lfloor \log_2 n \rfloor + 1) = (n^2/4 + n/2) \cdot (\lfloor \log_2 n \rfloor + 1)$$

El término dominante es $(n^2/4) \cdot \log n$; se descartan constantes y términos de menor orden:

$T(n) = O(n^2 \log n)$, y de hecho $\Theta(n^2 \log n)$, porque la expresión también está acotada por debajo.

b) Profiling

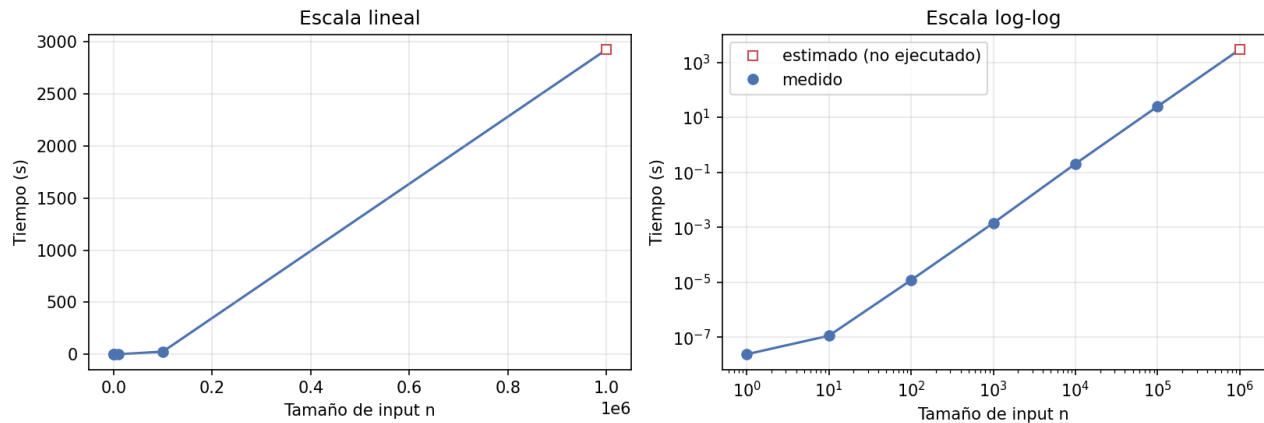
Método de profiling: reloj monótonico `clock_gettime(CLOCK_MONOTONIC)` en C (compilado con -O0). Para tamaños pequeños la función se repite hasta acumular 50 ms y se promedia. El `printf` se sustituyó por un contador `volatile` para medir el algoritmo y no la entrada/salida. Los contadores se verificaron contra las fórmulas exactas de iteraciones.

Nota: con $n = 1,000,000$ el programa realiza $\approx 5 \times 10^{12}$ iteraciones (unos 49 minutos). Esa fila se **estimó** como (operaciones exactas) $\times$ (ns por operación medidos en $n = 100,000$) en lugar de ejecutarla; está marcada como «estimado» en la tabla y con un cuadro vacío en la gráfica.

n	Operaciones	Tiempo	Origen
1	2	24.0 ns	medido
10	120	113.0 ns	medido
100	17,850	11.89 μ s	medido

n	Operaciones	Tiempo	Origen
1,000	2,505,000	1.42 ms	medido
10,000	350,070,000	203.70 ms	medido
100,000	42,500,850,000	24.89 s	medido
1,000,000	5,000,010,000,000	2928.13 s	estimado

Ejercicio 1 — $O(n^2 \log n)$



Análisis: en escala log-log la curva tiene pendiente ≈ 2 con una ligera curvatura hacia arriba, lo esperado para $n^2 \log n$. Cada vez que n se multiplica por 10, el tiempo se multiplica por ≈ 120 -140 (algo más de 100 por el factor $\log n$), consistente con $n^2 \cdot \log n$.

Ejercicio No. 2

```
void function (int n) {
    if (n <= 1) return;
    int i, j;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            printf ("Sequence\n");
            break;
        }
    }
}
```

a) Complejidad de tiempo

- La instrucción `if (n <= 1) return;` es $O(1)$.
- **Ciclo i:** ejecuta n iteraciones $\rightarrow \Theta(n)$.
- **Ciclo j:** aunque su condición permitiría n iteraciones, el `break` lo interrumpe al final de la **primera** iteración. Hace exactamente 1 iteración (un `printf` y el `break`), es decir, $O(1)$.

Como el ciclo interno cuesta $O(1)$ por cada valor de i :

$$T(n) = \sum_{i=1}^n O(1) = n \cdot c$$

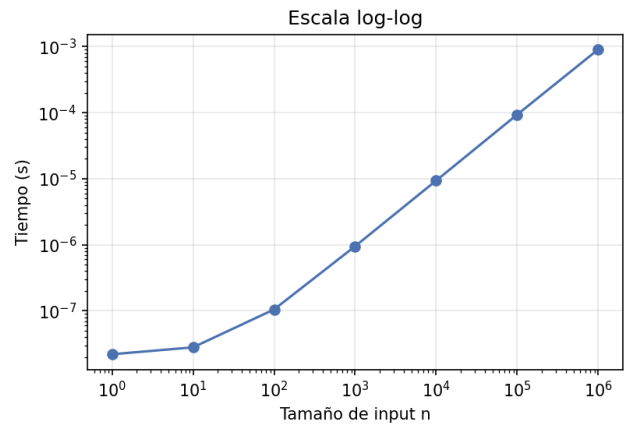
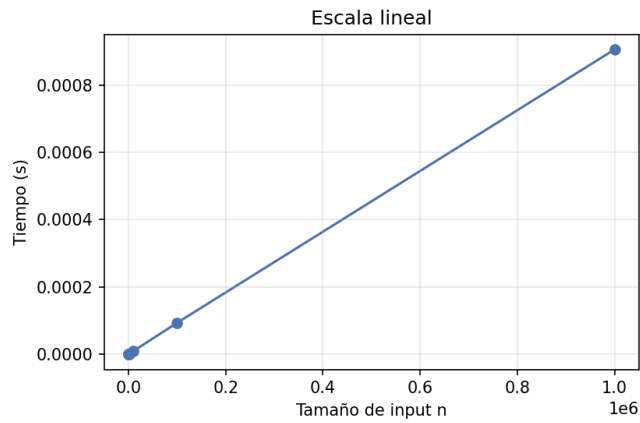
$T(n) = O(n)$ (y $\Theta(n)$ para $n > 1$). Es lineal y no cuadrático, pese a haber dos ciclos anidados.

b) Profiling

Método de profiling: reloj monotónico `clock_gettime(CLOCK_MONOTONIC)` en C (compilado con `-O0`). Para tamaños pequeños la función se repite hasta acumular 50 ms y se promedia. El `printf` se sustituyó por un contador `volatile` para medir el algoritmo y no la entrada/salida. Los contadores se verificaron contra las fórmulas exactas de iteraciones.

n	Operaciones	Tiempo	Origen
1	0	22.0 ns	medido
10	10	28.0 ns	medido
100	100	105.0 ns	medido
1,000	1,000	950.0 ns	medido
10,000	10,000	9.36 μ s	medido
100,000	100,000	93.40 μ s	medido
1,000,000	1,000,000	906.02 μ s	medido

Ejercicio 2 — $O(n)$



Análisis: el tiempo crece ≈ 10 veces por cada 10 veces que crece n : comportamiento lineal. Para $n = 1$ no hay trabajo (retorna de inmediato); el tiempo medido en esa fila es solo el costo de la llamada y del reloj.

Ejercicio No. 3

```
void function (int n) {  
    int i, j;  
    for (i=1; i<=n/3; i++) {  
        for (j=1; j<=n; j+=4) {  
            printf("Sequence\n");  
        }  
    }  
}
```

a) Complejidad de tiempo

- **Ciclo i:** va de 1 a $n/3$ con paso 1 $\rightarrow [n/3]$ iteraciones.
- **Ciclo j:** va de 1 a n con paso 4 ($j = 1, 5, 9, \dots$) $\rightarrow [n/4]$ iteraciones.

Las cotas son independientes, así que se multiplican:

$$T(n) = [n/3] \cdot [n/4] \approx (n/3) \cdot (n/4) = n^2/12$$

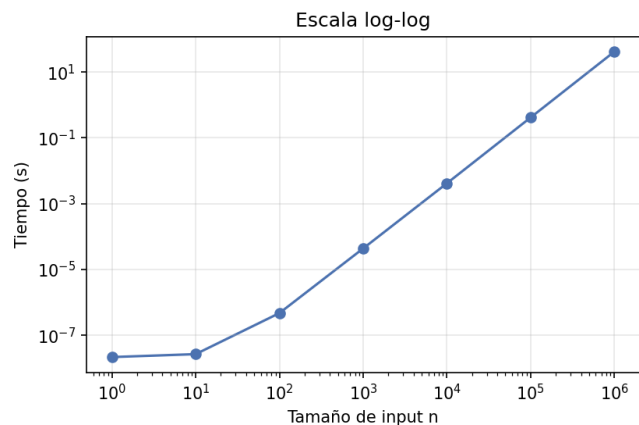
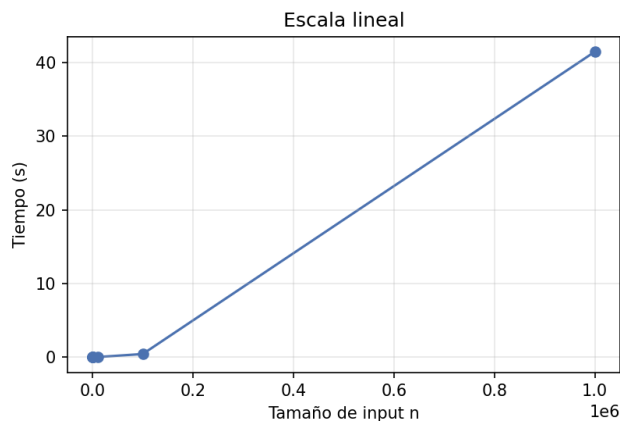
Se descarta la constante $1/12$: **$T(n) = O(n^2)$** (y $\Theta(n^2)$).

b) Profiling

Método de profiling: reloj monotónico `clock_gettime(CLOCK_MONOTONIC)` en C (compilado con `-O0`). Para tamaños pequeños la función se repite hasta acumular 50 ms y se promedia. El `printf` se sustituyó por un contador volátil para medir el algoritmo y no la entrada/salida. Los contadores se verificaron contra las fórmulas exactas de iteraciones.

n	Operaciones	Tiempo	Origen
1	0	22.0 ns	medido
10	9	27.0 ns	medido
100	825	476.0 ns	medido
1,000	83,250	43.70 μ s	medido
10,000	8,332,500	4.16 ms	medido
100,000	833,325,000	414.84 ms	medido
1,000,000	83,333,250,000	41.49 s	medido

Ejercicio 3 — $O(n^2)$



Análisis: cada vez que n se multiplica por 10, el tiempo se multiplica por ≈ 100 , como corresponde a un comportamiento cuadrático (pendiente 2 en escala log-log).

Ejercicio No. 4 — Búsqueda Lineal

```
LinearSearch(A, n, x):  
    for i = 0 to n-1:  
        if A[i] == x:  
            return i  
    return -1
```

Se cuenta el número de comparaciones $A[i] == x$ como operación dominante. Cada comparación es $O(1)$.

Mejor caso

El elemento buscado está en la primera posición ($A[0] = x$). Se hace 1 sola comparación: $T(n) = 1 \rightarrow \Theta(1)$ ($O(1)$).

Peor caso

El elemento está en la última posición o no está en el arreglo. Se comparan los n elementos: $T(n) = n \rightarrow \Theta(n)$ ($O(n)$).

Caso promedio

Se asume que x está en el arreglo y que su posición es equiprobable: la probabilidad de que esté en la posición k ($k = 1, \dots, n$) es $1/n$, y encontrarlo en esa posición cuesta k comparaciones. Entonces:

$$T_{\text{prom}}(n) = \sum_{k=1}^n k \cdot (1/n) = (1/n) \cdot n(n+1)/2 = (n+1)/2$$

Es decir, $\approx n/2$ comparaciones: $\Theta(n)$.

Si además se considera que x puede no estar con probabilidad q (costo n), el promedio es $(1-q)(n+1)/2 + q \cdot n$, que sigue siendo lineal en n . Por lo tanto el caso promedio también es $\Theta(n)$.

Resumen: mejor caso $\Theta(1)$, caso promedio $\Theta(n)$, peor caso $\Theta(n)$.

Ejercicio No. 5

a) Si $f(n) = \Theta(g(n))$ y $g(n) = \Theta(h(n))$, entonces $h(n) = \Theta(f(n))$. — VERDADERO

Θ es una relación de equivalencia (reflexiva, simétrica y transitiva). Por definición, $f = \Theta(g)$ significa que existen $c_1, c_2 > 0$ y n_0 tales que $c_1 g(n) \leq f(n) \leq c_2 g(n)$ para $n \geq n_0$; análogamente $d_1 h(n) \leq g(n) \leq d_2 h(n)$ para $n \geq n_1$.

Combinando, para $n \geq \max(n_0, n_1)$: $c_1 d_1 \cdot h(n) \leq f(n) \leq c_2 d_2 \cdot h(n)$, luego $f = \Theta(h)$ (transitividad).

Despejando, $(1/(c_2 d_2)) \cdot f(n) \leq h(n) \leq (1/(c_1 d_1)) \cdot f(n)$, así que $h = \Theta(f)$ (simetría).

b) Si $f(n) = O(g(n))$ y $g(n) = O(h(n))$, entonces $h(n) = \Omega(f(n))$. — VERDADERO

Por definición, $f = O(g)$ significa $f(n) \leq c \cdot g(n)$ para $n \geq n_0$, y $g = O(h)$ significa $g(n) \leq d \cdot h(n)$ para $n \geq n_1$. Entonces, para $n \geq \max(n_0, n_1)$: $f(n) \leq c \cdot d \cdot h(n)$, o sea $f = O(h)$ (transitividad de O).

Además, $h = \Omega(f)$ significa exactamente que existe una constante positiva e tal que $h(n) \geq e \cdot f(n)$ para n grande. Tomando $e = 1/(c \cdot d)$ se obtiene $h(n) \geq f(n)/(c \cdot d)$. Por lo tanto $h(n) = \Omega(f(n))$.

c) $f(n) = \Theta(n^2)$, con $f(n)$ el tiempo de ejecución de $A(n)$. — FALSO

```
def A(n):  
    atupla = tuple(range(0, n))  
    S = set()
```

```

for i in range(0, n):
    for j in range(i + 1, n):
        S.add(atupla[i:j])

```

El error es suponer que cada iteración del ciclo interno cuesta $O(1)$. Hay $n(n-1)/2$ pares (i, j) , pero en cada uno se hacen dos operaciones cuyo costo depende del tamaño de la rebanada:

- `atupla[i:j]` crea una tupla nueva copiando $j - i$ elementos $\rightarrow \Theta(j - i)$.
- `S.add(...)` calcula el hash de la tupla, que recorre sus $j - i$ elementos $\rightarrow \Theta(j - i)$. Si hay colisión, la comparación de igualdad también es proporcional a su longitud.

El costo total es:

$$f(n) = \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} \Theta(j - i)$$

Con $d = j - i$, para cada d hay $n - d$ pares, de modo que:

$$\sum_{d=1}^{n-1} (n - d) \cdot d = n \cdot n(n-1)/2 - (n-1)n(2n-1)/6 = (n^3 - n)/6 = \Theta(n^3)$$

Por lo tanto $f(n) = \Theta(n^3)$, y como n^3 crece estrictamente más rápido que n^2 , **$f(n) \neq \Theta(n^2)$** . El enunciado es falso.

Verificación empírica (`scripts/ejercicio5c.py`): el trabajo contado coincide con $(n^3 - n)/6$ (p. ej. $n = 800 \rightarrow 85,333,200$) y la razón tiempo/n^3 se estabiliza en $\approx 1.45 \times 10^{-9}$, mientras que tiempo/n^2 crece sin parar (de 1.1×10^{-7} a 1.2×10^{-6}).

n	tiempo (s)	trabajo	t / n ²	t / n ³
50	0.0003	20825	1.072e-07	2.143e-09
100	0.0018	166650	1.756e-07	1.756e-09
200	0.0114	1333300	2.852e-07	1.426e-09
400	0.0929	10666600	5.809e-07	1.452e-09
800	0.7444	85333200	1.163e-06	1.454e-09